Midterm Project: Scrapy for scraping

Group: №12

Professor: ……..……. /DINH-TRUNG VU (Assistant Professor) /

Student: ……/Ganbayar Gan-Erdene (113021202) /

Student: ……../Goo-Maral Ganzorig (113021200) /

113

Chapter 1: Introduction

1.1 GitHub

1. **Personal GitHub Account**
   My personal GitHub account is where I store my individual programming assignments, mini-projects, and coursework. For this project, I created and hosted a repository containing the Scrapy spider to extract information from my Github Profile.
2. Here is our group's repository.

1.2 Overview

In this project, I used **Scrapy**, a Python-based web scraping framework, to extract repository details from my GitHub profile page. The program collected:

- Repository URL
- Description (About)
- Last Updated time
- Programming Languages
- Number of Commits

Some of the advanced language features and libraries used in this project include:

- dataclass from the dataclasses module for structured data representation
- Regular expressions (re) for cleaning and formatting strings
- The Scrapy framework for crawling and data extraction
- Git for version control

The output was saved in an **XML file** format, containing structured information about each public repository from my GitHub profile.

Chapter 2: Implementation

2.1 Class 1: GitHubSpider (Scrapy Spider)

This spider class handles the crawling and parsing of the GitHub page.

2.1.1 Fields

- name: Name of the spider
- start_urls: The GitHub profile URL
- custom_settings: Defines the output format (XML) and location

2.1.2 Methods

- parse(): Scrapy's default method used to extract the list of repositories and their links
- parse_repo(): Navigates into each repository page to extract more detailed data like number of commits and languages used

2.1.3 Functions

- Regular expression functions were used to extract numeric values (e.g., commit counts)
- Helper functions cleaned and formatted extracted strings

2.2 Class 2

In this basic project, no second class was required. All logic was handled within the Scrapy spider.

2.3 Method/Function 1

The parse function processed the HTML structure of the profile page and scheduled further requests to repository pages.

2.4 Method/Function 2

The parse_repo function followed each repository URL and extracted data like languages used and last commit dates.

Chapter 3: Results

3.1 Result 1

The XML file (repos.xml) was generated successfully. It included structured records of all repositories, each with fields like name, URL, language, description, commits, and updated date.

3.2 Result 2

The spider correctly followed links, extracted dynamic content, and handled page structure changes by using XPath and CSS selectors carefully.

Chapter 4: Conclusions

This project helped me understand how to use Scrapy for real-world data collection. I learned to:

- Structure a Scrapy project
- Write and run spiders to crawl real web pages
- Use data classes and regular expressions for formatting
- Save results in a usable XML format
- Use Git and GitHub effectively for code versioning

It was a practical application of advanced Python programming concepts, and I feel more confident in applying these skills to future data projects.